

Language Server Protocol para YAP

Programação em Lógica
CC3012

2026

Docente: Vitor Costa

André Mendes up202307449
Eurico Magalhães up200701520

Conteúdos

Motivação	3
Language Server Protocol	3
pygls	4
Visual Studio Code ou Emacs	5
Configuração do sistema	6
Ambiente Virtual Python	7
YAP	7
LSP4YAP	8
Emacs	8
Utilizar lsp4yap	8
Funcionamento	9
lsp4yap	9
YAP	11

Motivação

Um dos possíveis trabalhos apresentados pelo professor foi o desenvolver uma solução para o seguinte problema: os utilizadores de editores de texto e IDE's esperam que estes programas possuam certas funcionalidades, como coloração de texto consoante a sintaxe da linguagem de programação, verificação e validação dessa sintaxe, procura de símbolos e definições, entre outras.

O compilador de Prolog YAP ainda não tem uma integração com possíveis soluções para este problema. Assim, este trabalho pretende investigar e implementar uma solução, usando o Language Server Protocol.

Language Server Protocol

O Language Server Protocol¹(LSP) é um protocolo de comunicação entre editores de texto e IDE's, e um servidor de uma linguagem de programação, que providencia ao editor as regras e instruções necessárias à implementação de funcionalidades de formatação e uso dessa linguagem no contexto do editor ou IDE.

Desenvolvido pela Microsoft, e *open source*, é um protocolo usado no Visual Studio Code²(VSCode) e, como este editor é bastante usado, atualmente, pelos alunos de Ciência de Computadores, o professor decidiu trabalhar neste contexto.

A motivação por trás do LSP, como explica a Microsoft, é a seguinte: em vez de cada editor de texto ter que implementar todas as linguagens de programação que pretende suportar, porque não ter um protocolo de comunicação que, quando implementado uma vez em cada editor, permite reconhecer todas as linguagens que implementem, por sua vez, esse protocolo.

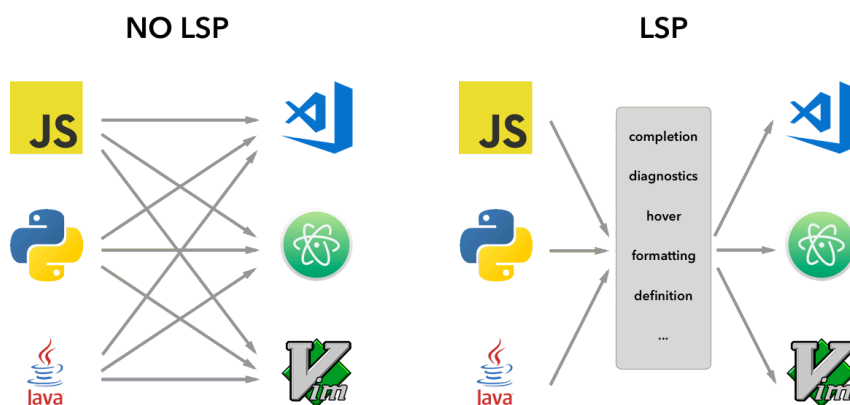


Figura 2: Motivação do LSP (fonte: site da API do VSCode²)

¹<https://microsoft.github.io/language-server-protocol/>

²<https://code.visualstudio.com/api/language-extensions/language-server-extension-guide>

Este protocolo implementa mensagens codificadas em JSON RPC³ que são trocadas entre cliente e servidor.

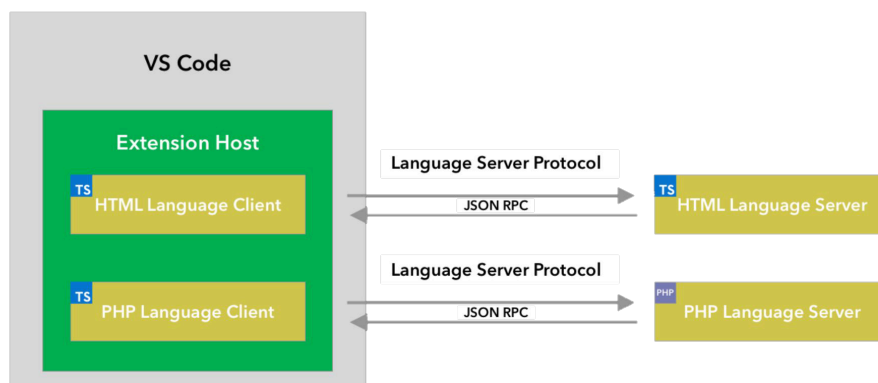


Figura 3: Funcionamento do LSP (fonte: site da API do VSCode²)

Como este protocolo se tornou popular, foram desenvolvidas várias ferramentas que permitem mais facilmente implementar um cliente/servidor LSP, evitando assim termos que, por exemplo, implementar todas as mensagens JSON.

Uma destas ferramentas é o pygls⁴.

pygls

O pygls é uma “implementação genérica do LSP escrita em Python”⁴. Usando o pygls deve ser possível escrever um servidor LSP em Python, de uma maneira relativamente simples.

Por exemplo, o código seguinte recebe um documento e, em cada linha que acabe em “hello.”, retorna ao editor de texto a hipótese de autocompletar com “world” ou “friend”, sempre que é introduzido um “.”.

³<https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>

⁴<https://pygls.readthedocs.io/en/latest/>

```

1 from pygls.lsp.server import LanguageServer
2 from lsprotocol import types
3
4 server = LanguageServer("example-server", "v0.1")
5
6
7 @server.feature(
8     types.TEXT_DOCUMENT_COMPLETION,
9     types.CompletionOptions(trigger_characters=["."]),
10    0 autocompletar só aparece quando se insere um "."
11 )
12 def completions(params: types.CompletionParams):
13     document =
14     server.workspace.get_text_document(params.text_document.uri)
15     current_line = document.lines[params.position.line].strip()
16
17     if not current_line.endswith("hello."):
18         return []
19
20     return [
21         types.CompletionItem(label="world"),
22         types.CompletionItem(label="friend"),
23     ]
24     Itens que aparecem no autocompletar do editor de texto
25
26 if __name__ == "__main__":
27     server.start_io()

```

O cliente dependerá, claro, do editor de texto/IDE. É aqui que surge o primeiro impedimento ao objetivo do trabalho.

Visual Studio Code ou Emacs

O VSCode⁵, nas palavras do professor, pode ser “bastante burocrático” na implementação de um cliente LSP. De facto, esse cliente teria de ser um projecto implementado em TypeScript, e o *debugging* envolve também muita burocracia, especialmente quando comparado com a alternativa: Emacs.

O Emacs⁶ é muito personalizável e, crucialmente, tem uma extensão que permite trabalhar com LSP, Eglot⁷. É relativamente fácil fazer com que esta extensão lance o nosso servidor, como veremos no próximo capítulo. Permite também monitorizar as mensagens enviadas entre Emacs e o servidor de uma maneira simples.

⁵<https://code.visualstudio.com/>

⁶<https://www.gnu.org/software/emacs/>

⁷<https://joaotavora.github.io/eglot/>

Configuração do sistema

No seu presente estado, este projecto precisa de três componentes:

- Um editor de texto/IDE. Usaremos Emacs, pelas razões mencionadas acima. Este editor de texto tem um cliente de LSP (Eglot);
- Um servidor de LSP. O professor providenciou o repositório `lsp4yap`, tanto no GitHub⁸ como no Codeberg⁹. Crucialmente, este repositório contém um ficheiro `yap.py`, que é o ponto de entrada do servidor. Este ficheiro, atualmente, recebe mensagens do editor (cliente LSP) e usa o próprio YAP para processar e assinalar com erros o texto do documento, enviando mensagens ao servidor `yap.py`, que depois as reenvia ao cliente;
- Um compilador de Prolog. Como vimos na entrada anterior, o nosso servidor precisa do YAP para funcionar. E obviamente faz sentido que, se estamos a escrever código em Prolog, temos um compilador dessa linguagem no mesmo sistema.

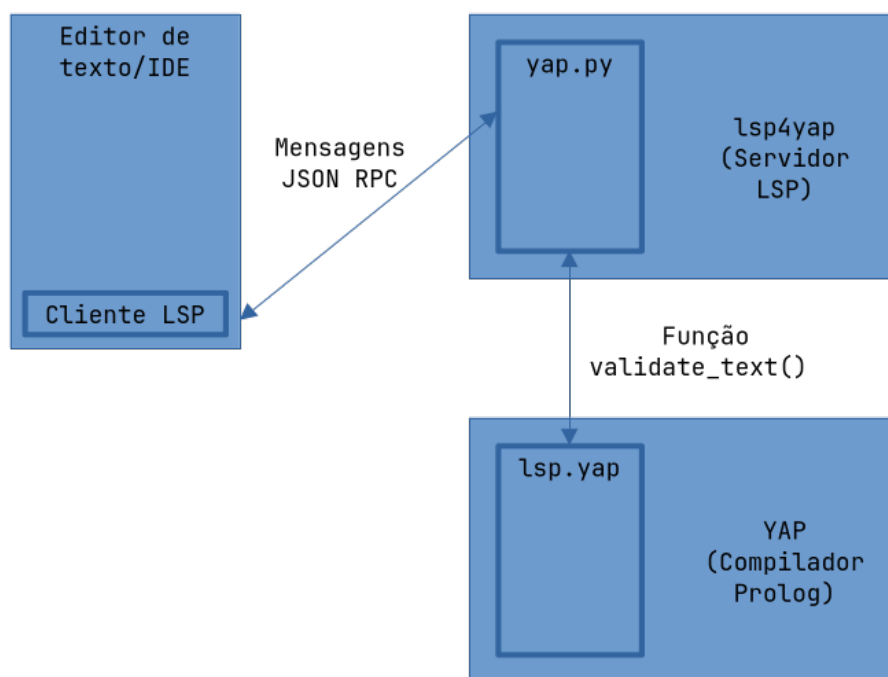


Figura 4: Visão global do projeto

Exploraremos os vários componentes com mais detalhe, mas primeiro preparemos o sistema.

Alguns caminhos importantes:

- `$ENV` é o caminho para um ambiente virtual Python, por exemplo `~/env`;
- `$LSP4YAP` é o caminho para o clone do repositório `lsp4yap`^{8,9};
- `$YAP` é o caminho para o clone do repositório `yap`^{10,11};

⁸<https://github.com/vscosta/lsp4yap>

⁹<https://codeberg.org/vscosta/lsp4yap>

Procedemos da seguinte maneira:

Ambiente Virtual Python

1. Criar um ambiente virtual Python. Navegar para o local pretendido (por exemplo ~/) e executar:

```
python3 -m venv env
```

bash

2. Este ambiente tem de ser ativado, para podermos usar os comandos python e pip a partir de \$ENV, e não do sistema. Se a shell usada for fish, o ficheiro a tocar é ligeiramente diferente:

```
source ./$ENV/bin/activate
```

bash

```
source ./$ENV/bin/activate.fish
```

fish

Para verificar se estamos a usar o ambiente virtual, executar os comandos seguintes e confirmar que o caminho devolvido é em \$ENV:

```
which python
which pip
```

bash

3. Podemos agora instalar as dependências Python:

```
pip install lsprotocol pygls numpy
```

bash

YAP

4. Clonar o repositório yap^{10,11} para \$YAP:

```
git clone <yap_URL>
```

bash

5. No diretório \$YAP, criar um diretório Build e navegar para aí:

```
mkdir Build
cd Build
```

bash

6. Compilar e instalar YAP, a partir de Build:

```
cmake ../
make
sudo make install
```

bash

¹⁰<https://github.com/vscosta/yap>

¹¹<https://codeberg.org/vscosta/yap>

7. Navegar para \$YAP e instalar o pacote Python yap4py no ambiente virtual (confirmar que está ativado, ponto 2):

bash

```
pip install packages/python/yap4py
```

LSP4YAP

8. Clonar o repositório lsp4yap^{8,9} para \$LSP4YAP:

bash

```
git clone <lsp4yap_URL>
```

Emacs

9. Instalar Emacs;
10. Configurar Emacs: no ficheiro de configuração, colocar:

lisp

```
(with-eval-after-load 'eglot
  (add-to-list 'eglot-server-programs
    '(prolog-mode . (" $ENV/bin/python" "$LSP4YAP/yap.py"))))

(add-hook 'prolog-mode-hook 'eglot-ensure)

(setq auto-mode-alist (append '(("\\.pl\\\\" . prolog-mode)
                                ("\\.yap\\\\" . prolog-mode))
                              auto-mode-alist))
```

Utilizar lsp4yap

11. No Emacs, abrir um ficheiro de Prolog e fazer M-x eglot (Alt+X, escrever eglot, pressionar Enter)

Funcionamento

lsp4yap

Atentemos agora com algum cuidado ao diretório `lsp4yap`.

O ficheiro `yap.py` é o ponto de entrada do nosso servidor. De notar que, no ficheiro `packages.json`, alguma variáveis devem ser definidas. Por exemplo:

json

```
1 "configuration": [  
2   {  
3     "type": "object",  
4     "title": "Server Configuration",  
5     "properties": {  
6       "pygls.server.launchScript": {  
7         "scope": "resource",  
8         "type": "string",  
9         "default": "examples/servers/yap.py",  
10        "description": "The python script to run when launching the  
server.",  
11        "markdownDescription": "The python script to run when launching  
the server.\nRelative to #pygls.server.cwd#"  
12      }  
13    }  
14  }  
15 ]
```

Define que o servidor `pygls` deve correr `yap.py` quando inicia.

É importante recordar que, na configuração do Emacs, nós definimos que a extensão Eglot deve correr o ficheiro `yap.py` quando entra no modo Prolog, “ignorando” assim estas configurações. Esta é mais uma razão para ter escolhido Emacs, a facilidade de ligar o editor de texto com o ficheiro que especificamos diretamente. Os ficheiros de configuração envoltantes que o `pygls` providencia são úteis quando quisermos adaptar este projeto para Visual Studio Code.

No ficheiro yap.py, uma função importante é validate():

python

```
1 def validate(self, document: TextDocument):
2     """Validates prolog file."""
3
4     diagnostics = []
5     uri = document.uri
6     print(" Master!!!!!!!!!!!!", file = sys.stderr)
7     data = engine.fun(validate_text(uri,document.source))
8     print(" Master:vdone")
9
10    errs = data
11
12    if errs:
13        for (sev,msg,i0,j0,i1,j1) in errs:
14
15            if sev == "warning":
16                sev=types.DiagnosticSeverity.Warning
17            else:
18                sev=types.DiagnosticSeverity.Error
19
20            location=types.Range(
21                start=types.Position(line=i0-1, character=j0-1),
22                end=types.Position(line=i1-1, character= j1-1)
23            )
24
25            diagnostics.append(
26                types.Diagnostic(
27                    message = msg,
28                    severity=types.DiagnosticSeverity.Warning,
29                    range = location
30                )
31            )
32
33    return document.version,diagnostics
```

Este array de Diagnostics é preenchido com os vários erros

A função validate_text() interage com o YAP, processando assim o documento num contexto de prolog, e recebendo como retorno erros, que são guardados em data.

Esse array de erros é depois processado aqui, num contexto de pygls. Range, Position, Diagnostic, entre outras, são construções que o pygls usa para definir as mensagens JSON RPC trocadas entre cliente e servidor LSP.

Aqui construímos os vários Diagnostic, para cada erro reportado. Usamos os valores retornados pela função YAP validate_text() (sev, msg, e valores de linha e coluna).

YAP

Ainda no ficheiro `yap.py`, a função `validate_text()` interage com `engine`, que é inicializado da seguinte maneira:

python

```
1 from yap4py.yapi import Engine, EngineArgs
2
3 # [...] #
4
5 def start_yap():
6     eargs.jupyter = True
7     try:
8         engine = Engine( eargs)
9         engine.load_library("lsp")
10    except Exception:
11        print("bad load")
12        engine = None
13    return engine
```

`yap4py.yapi`, em `$YAP/packages/python`, funciona como um interpretador de Prolog dentro do YAP, capaz de receber instruções em Python e corrê-las em Prolog.

Desta maneira, usamos o compilador YAP como um interpretador de texto Prolog que, em vez de converter esse texto para código máquina (como o faria em “modo de compilador”), reporta apenas erros sintáticos e informa-nos acerca de tabelas de símbolos, as funcionalidades que requeremos para implementar um LSP.

Assim, voltando a `yap.py`, a linha:

python

```
data = engine.fun(validate_text(uri,document.source))
```

É processada pelo `engine`, que é um “interpretador”, e corre a função `validate_text(uri,document.source)`. Consegue fazê-lo porque importa a biblioteca `lsp`, que é definida no ficheiro `lsp.yap`, também em `$YAP/packages/python`. Dentro deste ficheiro está definido o predicado correspondente, `validate_text/3`:

prolog

```
1 validate_text(URI,S,Ts) :-
2
3     writeln(user_error, URI),
4
5     string_concat("file://", FileAsS, URI),
6     atom_string(FileAsS, File),
7
8     retractall(def(_,_,_,File,_)),
9     retractall(use(_,_,_,_,_,File,_)),
10    retractall(dec(_,_,_,File,_)),
11    Apaga dados anteriores.
```

```

12     open(string(S),read,Stream),
13     current_source_module(DefaultModule,user),
    Abre o texto como uma Stream.
14
15     assert(lsp(on)),
    Liga o "modo" LSP.
16
17     load_files(File,[stream(Stream)]),
18     current_source_module(_,DefaultModule),
    Carrega e processa a Stream de texto.
19
20     retractall(lsp(_)),
    Desliga o "modo" LSP.
21
22     findall(T,retract(m(T)),Ts).
    Recolhe todos os erros.

```

Em Prolog, existem predicados chamados *hooks* que são corridos automaticamente pelo YAP¹². Assim, a linha `load_files(File,[stream(Stream)])` ativa o hook `term_expansion/2`, que é (re)definido neste ficheiro:

prolog

```

1 user:term_expansion(G, GF) :-
2
3     lsp(on),
    Só corre se estivermos no "modo" LSP
4
5     prolog_load_context(file, F ),
6     prolog_load_context(term_position, Pos ),
7     current_source_module(M, M ),
8     arg(2, Pos, Line),
9
10    analyse(G,Line,F,M,GF),
    Analisa o termo Prolog
11    !.

```

`analyse/5` descobre que tipo de termo Prolog estamos a tratar:

prolog

```

1 analyse((:- G), L, F, M, (:- true)) :-
2     !,
3     directive(G, L, F, M).
    Cláusulas que só têm corpo.
4
5 analyse((:- _G), _L, _F, _M, (:- true)) :-
6     !.
    Outras cláusulas só com corpo.
7
8 analyse(Cl, L, F, M, Na) :-

```

¹²<https://www.swi-prolog.org/pldoc/man?section=hooks>

```

9      rule(Cl, L, F, M, Na).
      Cláusulas de Factos e Regras, a maioria do casos.
10
11 analyse(_G, _L, _F, _M, Na) :-
12     strip_module(G,_MH,A),
13     functor(A,Na,_Ar).
      O resto que não for apanhado pelos casos anteriores.

```

O caso mais comum é uma Cláusula de Factos ou Regras. Relembremos que Factos são do tipo:

```
p(a).
```

prolog

E Regras são:

```
q(a) :- p(a).
```

prolog

Portanto, para explorar estes casos, usamos o predicado rule/5:

```

1 rule((A0:- B),Line,File,Module,Na) :-
2     strip_module(Module:A0,MH,A),
3     functor(A,Na,Ar),
4     (
5     def(Na,Ar,MH,_,_,predicate)
6     ->
7     true
8     ;
9     assert(def(Na,Ar,MH,Line,File,predicate))
10    ),
11    body(B,Line,File,Module,MH:Na/Ar).

```

prolog

O predicado seguinte, que também é um hook, corre quando o YAP deteta um erro¹³. Neste caso, term_expansion/2 não é chamado, mas sim este predicado:

```

1 user:portray_message(A,B):-
2     lsp(on),
3     !,
4     writeln(user_error,B),
5     (
6     q_msg(A,B,T),
      Converte a mensagem num tuplo
7     A \= "ignored"
8     ->
9     assert(m(T))
      Guarda esse tuplo na base de dados
10    ;
11    true

```

prolog

¹³<https://sicstus.sics.se/sicstus/docs/3.12.10/html/sicstus/Message-Handling-Predicates.html>

12).

q_msg/3 filtra os vários tipos de mensagens geradas pelo YAP, e devolve as que são úteis ao LSP em tuplos t(Severity, Message, StartLine, StartColumn, EndLine, EndColumn).
prolog

```
1 q_msg(informational, _, _) :-
2     !,
3     fail.
4
5 q_msg(help, _, _) :-
6     !,
7     fail.
8
9 Mensagens informacionais ou de ajuda são ignoradas.
10
11 q_msg(warning, error(style_check(singletons,
12     [VName,Line,Column,_F0],_),_Desc),t("warning",S, Line,Column,
13     Line,EndCol)) :-
14     !,
15     format(string(S), 'singleton variable ~s.~n ', [VName]),
16     atom_length(VName, Len),
17     EndCol is Column+Len.
18
19 Um aviso que reporta erros de singletons é formatado num tuplo de tipo
20 "warning", com a string definida aqui, e a sua localização em linhas e
21 colunas.
22
23 q_msg(warning, error(style_check(multiple,[F0|L],I ), _Desc ),
24     t("warning",S, L,Column,L,EndCol)) :-
25     !,
26     Column=1,
27     format(string(S), '~w previously defined at ~s.~n',[I,F0]),
28     I = Name/_,
29     atom_length(Name, Len),
30     EndCol is Column+Len.
31
32 O mesmo predicado definido múltiplas vezes.
33
34 q_msg(warning, error(style_check(discontiguous,_,I ), _Desc),
35     t("warning",S, L,Column, L, EndCol)) :-
36     !,
37     Column=1,
38     format(string(S), 'discontiguous definition for ~w.~n',[I]),
39     I = Name/_,
40     atom_length(Name, Len),
41     EndCol is Column+Len,
42     S = "discontiguous.~n".
43
44 Cláusulas do mesmo predicado que não estão agrupadas.
45
46 q_msg(_error, error(syntax_error(_Msg), Desc), t("error","syntax error",
47     L,LPos,L,LPl)) :-
```

```
33     exception_property(parserLine, Desc, L),
34     exception_property(parserLinePos, Desc, LPos),
35     exception_property(parserSize, Desc, Size),
36     !,
37     LP1 is LPos+Size.
```

Erros sintáticos.

```
38
39 q_msg(_,Error,t("ignored",Msg,1,0,2,0)) :-
40     term_to_string(Error,Msg).
```

Quaisquer outros erros.